

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA Nº 06

NOMBRE COMPLETO: José Eduardo Martínez García

Nº de Cuenta: 320304679

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 05

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 07/10/2026

CALIFICACIÓN: _____

Reporte de práctica 6: Optimización y Carga de Modelos

1. Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

a) Ejercicio 1: Crear o modificar el holocron de código y texturizarlo por medio de código (Dice modificar porque pueden o modificar las coordenadas que ya les di en el main, u obtenerlas de un nuevo modelo de Holocron que hagan ustedes por ejemplo en Geogebra o Blender).

Se modifico la función de crear holocron para realizar la parte del texturizado por medio de código, se agregaron las secciones de U y V para poner mapear las coordenadas de cada uno de los símbolos/logos de StarWars en un rango de [0,1] de tal manera que quedaran lo mas similar al modelo del holocron texturizado en blender e importado a la ventana de la escena.

El código de esta función modificada es:

```
void CrearHolocron()
{
	GLfloat holocron_vertices[] = {

		// --- PARTE CARAS CUADRADAS ---

		// x	y	z	S	T	NX	NY	NZ
		-2.500f,  2.500f,  0.000f,	0.0000f, 0.8000f,	0.0f,	1.0f,	0.0f,
		 0.000f,  2.500f,  2.500f,	0.1666f, 0.8000f,	0.0f,	1.0f,	0.0f,
		 2.500f,  2.500f,  0.000f,	0.1666f, 1.0000f,	0.0f,	1.0f,	0.0f,
		 0.000f,  2.500f, -2.500f,	0.0000f, 1.0000f,	0.0f,	1.0f,	0.0f,

	-2.500f, -2.500f,  0.000f,	0.1666f, 0.8000f,	0.0f,	-1.0f,	0.0f,
		 0.000f, -2.500f, -2.500f,	0.3333f, 0.8000f,	0.0f,	-1.0f,	0.0f,
		 2.500f, -2.500f,  0.000f,	0.3333f, 1.0000f,	0.0f,	-1.0f,	0.0f,
		 0.000f, -2.500f,  2.500f,	0.1666f, 1.0000f,	0.0f,	-1.0f,	0.0f,



		-2.500f,  0.000f,  2.500f,	0.3333f, 0.8000f,	0.0f,	0.0f,	1.0f,
		 0.000f, -2.500f,  2.500f,	0.5000f, 0.8000f,	0.0f,	0.0f,	1.0f,
		 2.500f,  0.000f,  2.500f,	0.5000f, 1.0000f,	0.0f,	0.0f,	1.0f,
	0.000f,  2.500f,  2.500f,	0.3333f, 1.0000f,	0.0f,	0.0f,	1.0f,

		 2.500f,  0.000f, -2.500f,	0.5000f, 0.8000f,	0.0f,	0.0f,	-1.0f,
		 0.000f, -2.500f, -2.500f,	0.6666f, 0.8000f,	0.0f,	0.0f,	-1.0f,
		-2.500f,  0.000f, -2.500f,	0.6666f, 1.0000f,	0.0f,	0.0f,	-1.0f,
	0.000f,  2.500f, -2.500f,	0.5000f, 1.0000f,	0.0f,	0.0f,	-1.0f,

		 2.500f, -2.500f,  0.000f,	0.6666f, 0.8000f,	1.0f,	0.0f,	0.0f,
		 2.500f,  0.000f,  2.500f,	0.8333f, 0.8000f,	1.0f,	0.0f,	0.0f,
		 2.500f,  2.500f,  0.000f,	0.8333f, 1.0000f,	1.0f,	0.0f,	0.0f,
		 2.500f,  0.000f, -2.500f,	0.6666f, 1.0000f,	1.0f,	0.0f,	0.0f,
```

```
        -2.500f, -2.500f,  0.000f,    0.8333f, 0.8000f,  -1.0f,   0.0f,   0.0f,
        -2.500f,  0.000f, -2.500f,    1.0000f, 0.8000f,  -1.0f,   0.0f,   0.0f,
        -2.500f,  2.500f,  0.000f,    1.0000f, 1.0000f,  -1.0f,   0.0f,   0.0f,
        -2.500f,  0.000f,  2.500f,    0.8333f, 1.0000f,  -1.0f,   0.0f,   0.0f,

                        // --- PARTE DE CARAS TRIANGULARES ---

        2.500f,  2.500f,  0.000f,    0.0000f, 0.6000f,   0.577f, 0.577f, 0.577f,
        2.500f,  0.000f,  2.500f,    0.1666f, 0.6000f,   0.577f, 0.577f, 0.577f,
        0.000f,  2.500f,  2.500f,    0.0833f, 0.8000f,   0.577f, 0.577f, 0.577f,

        -2.500f,  0.000f,  2.500f,    0.1666f, 0.6000f,  -0.577f, 0.577f, 0.577f,
        -2.500f,  2.500f,  0.000f,    0.3333f, 0.6000f,  -0.577f, 0.577f, 0.577f,
        0.000f,  2.500f,  2.500f,    0.2500f, 0.8000f,  -0.577f, 0.577f, 0.577f,

        0.000f, -2.500f,  2.500f,    0.3333f, 0.6000f,   0.577f,-0.577f, 0.577f,
        2.500f, -2.500f,  0.000f,    0.5000f, 0.6000f,   0.577f,-0.577f, 0.577f,
        2.500f,  0.000f,  2.500f,    0.4166f, 0.8000f,   0.577f,-0.577f, 0.577f,

        -2.500f, -2.500f,  0.000f,    0.5000f, 0.6000f,  -0.577f,-0.577f, 0.577f,
        0.000f, -2.500f,  2.500f,    0.6666f, 0.6000f,  -0.577f,-0.577f, 0.577f,
        -2.500f,  0.000f,  2.500f,    0.5833f, 0.8000f,  -0.577f,-0.577f, 0.577f,

    2.500f,  0.000f, -2.500f,    0.6666f, 0.6000f,   0.577f, 0.577f,-0.577f,
    2.500f,  2.500f,  0.000f,    0.8333f, 0.6000f,   0.577f, 0.577f,-0.577f,
        0.000f,  2.500f, -2.500f,    0.7500f, 0.8000f,   0.577f, 0.577f,-0.577f,


        -2.500f,  2.500f,  0.000f,    0.8333f, 0.6000f,  -0.577f, 0.577f,-0.577f,
        -2.500f,  0.000f, -2.500f,    1.0000f, 0.6000f,  -0.577f, 0.577f,-0.577f,
        0.000f,  2.500f, -2.500f,    0.9166f, 0.8000f,  -0.577f, 0.577f,-0.577f,

        2.500f, -2.500f,  0.000f,    0.0000f, 0.4000f,   0.577f,-0.577f,-0.577f,
        0.000f, -2.500f, -2.500f,    0.1666f, 0.4000f,   0.577f,-0.577f,-0.577f,
        2.500f,  0.000f, -2.500f,    0.0833f, 0.6000f,   0.577f,-0.577f,-0.577f,

        0.000f, -2.500f, -2.500f,    0.1666f, 0.4000f,  -0.577f,-0.577f,-0.577f,
        -2.500f, -2.500f,  0.000f,    0.3333f, 0.4000f,  -0.577f,-0.577f,-0.577f,
        -2.500f,  0.000f, -2.500f,    0.2500f, 0.6000f,  -0.577f,-0.577f,-0.577f
};

unsigned int holocron_indices[] = {
        // Cuadrados
        0,  1,  2,   2,  3,  0,
        4,  5,  6,   6,  7,  4,
        8,  9,  10,  10, 11, 8,
        12, 13, 14,  14, 15, 12,
        16, 17, 18,  18, 19, 16,
        20, 21, 22,  22, 23, 20,
```

```
                // Triángulos
                24, 25, 26,
                27, 28, 29,
                30, 31, 32,
                33, 34, 35,
                36, 37, 38,
                39, 40, 41,
                42, 43, 44,
                45, 46, 47
        };

        MeshModel* holocron = new MeshModel();
        holocron->CreateMeshModel(holocron_vertices, holocron_indices, 384, 60);
        meshListModel.push_back(holocron);
}
```

La salida lograda después de texturizar el holocron propuesto mediante código en c++ es la siguiente:



Imagen 1. Salida del holocron por código en Open GL

b) Ejercicio 2: Importar el modelo del holocron texturizado en el programa de modelado

Se importo el modelo del holocron dentro de blender y después se realizo el texturizado ya con la imagen bien escalada a en una proporción 2n x 2n, por lo que finalmente la imagen tuvo una escala de 520 x 520 y esta fue la imagen con la que se mapearon las caras del holocron.

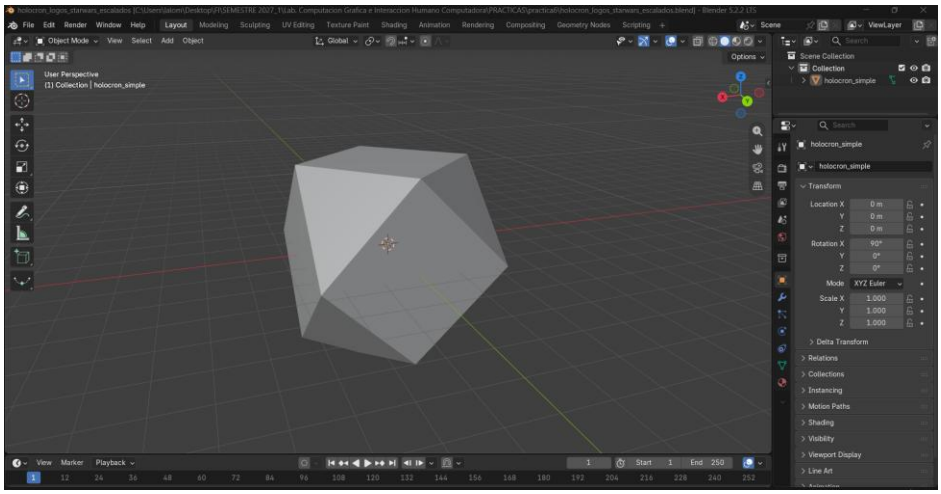


Imagen 2. Importación del modelo del holocron en Blender

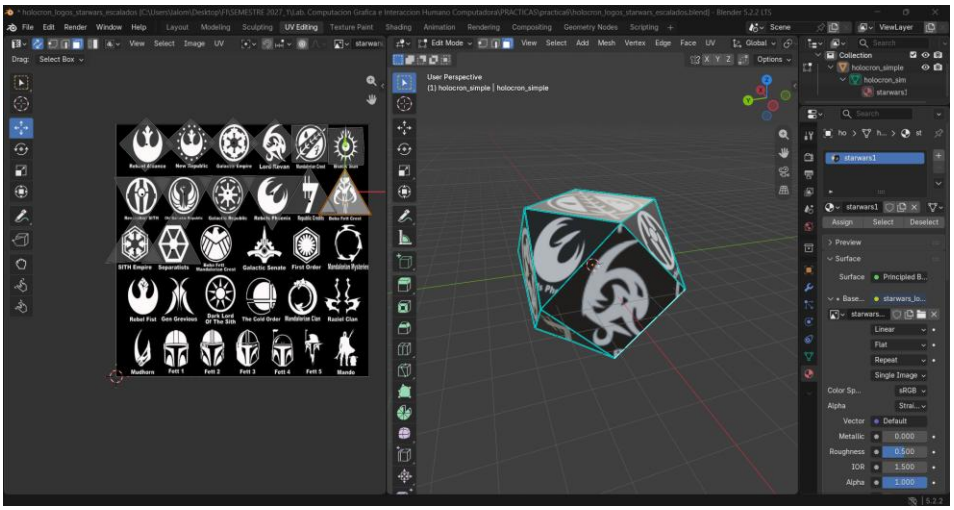


Imagen 3. Texturamiento del holocron en Blender

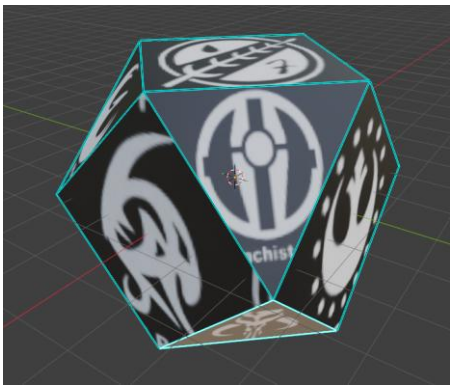


Imagen 4. Holocron texturizado

Se definieron variables para las texturas:

```
Texture holocronTextureStarWars;
Texture avionTexture;
```

Se definieron variables para los modelos:

```
Model Holocron_starwars;
Model avion;
```

Dentro del “Main” se cargaron los modelos manipulados y texturizados desde blender:

```
	Holocron_starwars = Model();
	Holocron_starwars.LoadModel("Models/holocron_logos_starwars_escalados.obj"
);

	avion = Model();
	avion.LoadModel("Models/avion_texturizado.obj");
```

Se importo en la escena dentro del “ while (!mainWindow.getShouldClose())” con el siguiente código:

```
//>>>>Holocron con texturizado de logos de StarWars desde Blender>>>>

		color = glm::vec3(1.0f, 1.0f, 1.0f);
		glUniform3fv(uniformColor, 1, glm::value_ptr(color));
		model = glm::mat4(1.0);
		model = glm::translate(model, glm::vec3(5.0f, 2.0f, -2.0f));
		model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
		glUniformMatrix4fv(uniformModel, 1, GL_FALSE,
glm::value_ptr(model));
		holocronTextureStarWars.UseTexture();
		Holocron_starwars.RenderModel();
```

Salida del modelo dentro de la escena en la ventana de Open GL:



Imagen 5. Holocron texturizado dentro de OpenGL

c) Ejercicio 3: Importar un modelo de avión con la textura de la cara del personaje de la imagen del previo:

- Vidrio frontal: OJO.
- Frente del avión: Nariz y Sonrisa
- Alas: Logos del universo del personaje

Se descargo el modelo del avión en la pagina de cgtrader. El modelo se puede acceder en el siguiente link: https://www.cgtrader.com/items/4025533/download-page

Venia con un conjunto de aviones del mismo modelo, entonces lo que se realizo fue eliminar los excedentes y dejar solamente el avión que se iba a utilizar para el modelado:

De la imagen referencia se obtuvo la siguiente imagen con la que se realizo la parte del texturizado posteriormente:

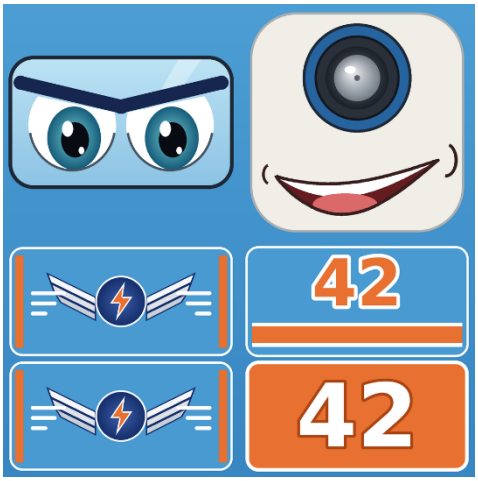


Imagen 6. Imagen optimizada y escalada del avión de referencia

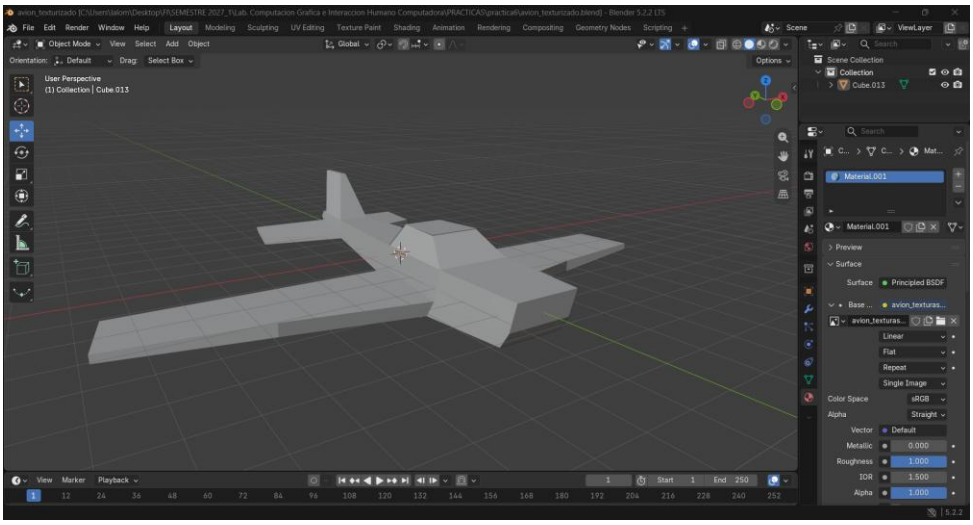


Imagen 7. Importación del modelo del avión en Blender

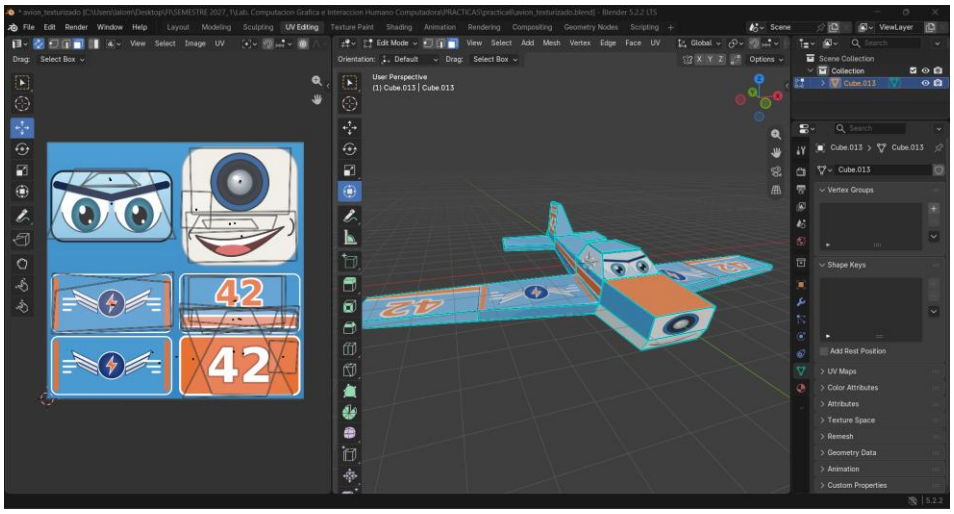


Imagen 8. Texturizado del avión en Blender

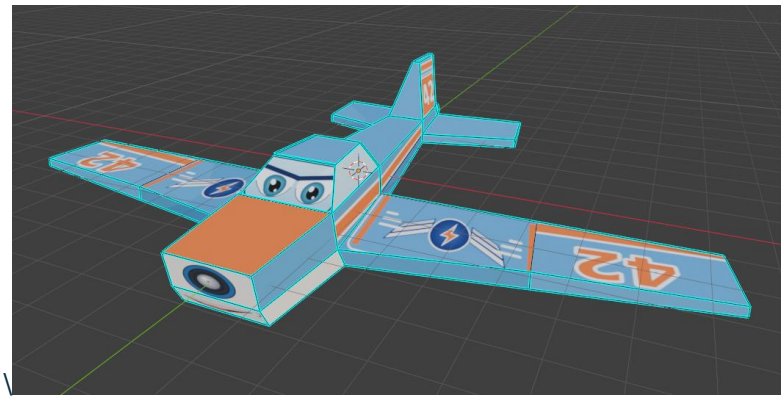


Imagen 9. Avion texturizado

Se definieron variables para las texturas:

```
Texture avionTexture;
```

Se definieron variables para los modelos:

```
Model avion;
```

Dentro del “Main” se cargaron los modelos manipulados y texturizados desde blender:

```
	avion = Model();
	avion.LoadModel("Models/avion_texturizado.obj");
```

Se importo en la escena dentro del “ while (!mainWindow.getShouldClose())” con el siguiente código:

```
//>>>>>>Avion con texturizado de logos de StarWars desde Blender>>>>>

		color = glm::vec3(1.0f, 1.0f, 1.0f);
		glUniform3fv(uniformColor, 1, glm::value_ptr(color));
		model = glm::mat4(1.0);
		model = glm::translate(model, glm::vec3(5.0f, 2.0f, -22.0f));
		model = glm::scale(model, glm::vec3(1.5f, 1.5f, 1.5f));
		glUniformMatrix4fv(uniformModel, 1, GL_FALSE,
glm::value_ptr(model));
		avionTexture.UseTexture();
		avion.RenderModel();
		holocronTextureStarWars.UseTexture();
		Holocron_starwars.RenderModel();
```

Salida del modelo dentro de la escena en la ventana de Open GL:

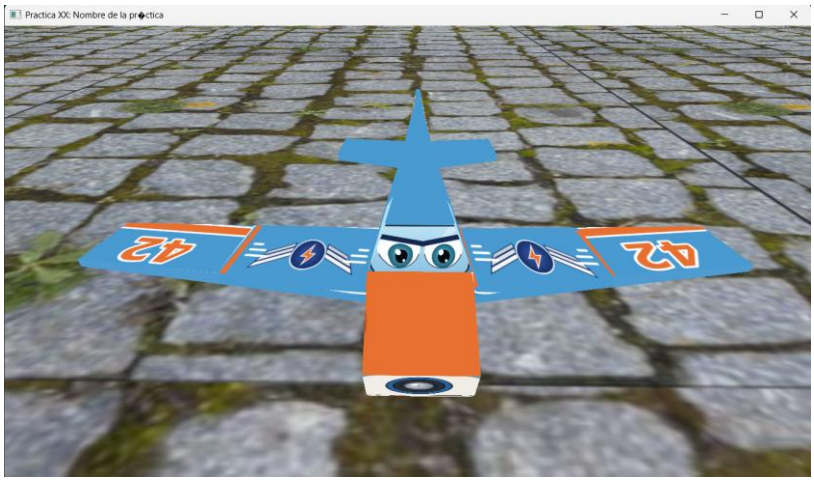


Imagen 10. Avion Texturizado dentro de la escena de Open GL

2.Problemas presentados a la hora de resolver los ejercicios:

- Al principio me costó mucho entender la lógica detrás de las coordenadas S y T (U y V) y para qué servían las normales. No me quedaba claro cómo es que el código sabía qué parte de la imagen poner en cada cara del cubo con solo darle unos puntos. Lo solucioné al comprender que los vértices actúan como "tachuelas" que estiran un porcentaje de la imagen entre ellos, y que las normales solo indican hacia dónde "mira" la cara para que la iluminación no se vea negra

- En Blender se me complicó la limpieza de los modelos antes de texturizar. A veces la herramienta automática para unir los triángulos y hacer caras cuadradas unía cosas que no debía. Lo solucioné haciéndolo de forma manual, seleccionando los pares de triángulos en el Modo Edición y uniéndolos con la tecla F. También me costó entender cómo mapear la imagen, pero lo resolví usando la herramienta "Unwrap" seleccionando cara por cara para acomodar los logos usando un solo material.

- Al importar los modelos a OpenGL tuve un par de errores visuales: primero los modelos salían de color negro o tintados de verde brillante. Lo arreglé asegurándome de regresar el multiplicador de color a blanco en el código antes de dibujar y marcando la casilla de "Exportar Normales" en Blender. Después tuve un problema de "estática" con rayas diagonales de colores en la textura; lo solucioné dándome cuenta de que la función del código esperaba una imagen con canal de transparencia (RGBA) y mi imagen TGA no lo tenía, así que se lo agregué desde GIMP.

3.Conclusión:

a) Lo más importante de esta práctica fue entender el proceso real del texturizado 3D. Aprendí que la forma más eficiente y correcta de texturizar no es crear un material o archivo de imagen distinto para cada cara de la figura, sino usar un "Atlas de texturas" (una sola imagen cuadrada con todos los logos o colores pegados) y usar el mapeo UV para indicarle al programa qué recorte de la imagen le toca a cada polígono.

b) El flujo de trabajo entre los distintos programas fue laborioso pero clave para que la práctica saliera bien. Entendí que el archivo .obj que sale de Blender es solo un texto que guarda la geometría y el mapa UV, pero no guarda la imagen en sí. Por eso es vital cargar la imagen .png o .tga manualmente en la memoria usando C++ y activarla antes de renderizar el modelo.

c) Se cumplieron todos los ejercicios: se logró texturizar el holocrón por código calculando las coordenadas para mapear los 14 logos de Star Wars de forma independiente, y se importaron con éxito los modelos del holocrón y el avión texturizados desde Blender. Se solucionaron los detalles de geometría, los problemas de lectura de bytes de las imágenes y la iluminación de las normales para que la escena cargara perfectamente.